Lecture 26: Loops in MIPS Assembly Language (`while`, `for` & `do-while`)

🎯 Learning Objectives

By the end of this lecture, students will be able to:

- Understand the concept of loops in Assembly Language.
- Implement `while`, `for`, and `do-while` loops in MIPS.
- Translate C/C++ loop code into MIPS.
- Use branch and jump instructions to control loop execution.
- Write practical looping programs in MARS.
- Debug loops step-by-step.

---

1. Introduction

🎤 Teaching Script

First 30-Second SEO Phrase

Naturally mention:

> **"In today's Computer Organization and Assembly Language lecture, we'll learn how to implement `while`, `for`, and `do-while` loops in MIPS Assembly Language using comparison, branch, and jump instructions, with practical programming examples in the MARS simulator."**

In the previous lecture, we learned how MIPS programs make decisions using comparison, branch, and jump instructions.

But in programming, we often need to repeat the same task many times.

For example:

- Print the numbers from 1 to 10.
- Calculate the sum of the first 100 numbers.
- Display the multiplication table of a number.

Instead of writing the same instructions again and again, we use loops.

In today's lecture, we'll learn how `while`, `for`, and `do-while` loops are implemented in MIPS Assembly Language using labels, comparison instructions, branches, and jumps.

By the end of this lecture, you'll be able to translate loop-based C/C++ programs into MIPS Assembly.

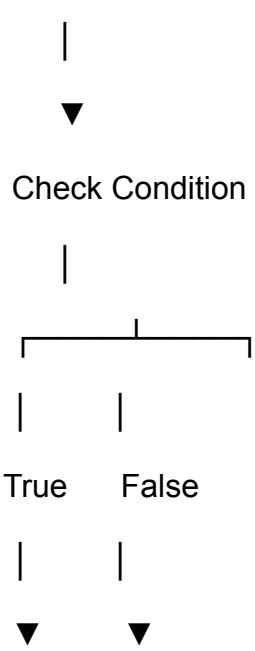
Let's begin.

---

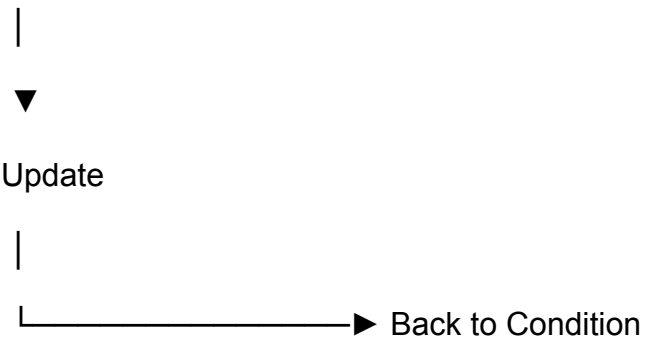
2. What is a Loop?

A **loop** is a programming construct that repeatedly executes a block of code until a condition becomes false.

General flow:



Loop Body Exit



---

3. Components of Every Loop

Every loop has four parts:

1. **Initialization**
2. **Condition**
3. **Loop Body**
4. **Update**

Example in C++:

```
for(int i = 1; i <= 5; i++)

{

   cout << i;

}
```

---

4. The `while` Loop

C++ Code

```
int i = 1;


while(i <= 5)
{
    cout << i;
    i++;
}
```

MIPS Translation

```
li $t0, 1          # i = 1


While:

    slti $t1, $t0, 6
    beq  $t1, $zero, EndWhile

    # Loop Body

    addi $t0, $t0, 1

    j While
```

EndWhile:

Explanation

$t0 → i

slti → Check i < 6

beq → Exit loop if false

addi → Increment i

j → Repeat loop

---

5. Printing Numbers 1–5

.text

.globl main

main:

```
li $t0,1

Loop:

    slti $t1,$t0,6

    beq $t1,$zero,Exit

    li $v0,1
    move $a0,$t0
    syscall

    li $v0,11
    li $a0,10
    syscall

    addi $t0,$t0,1

    j Loop

Exit:

li $v0,10
```

syscall

Output:

1

2

3

4

5

---

6. The `for` Loop

General C++ Syntax:

```
for(initialization; condition; update)
{
    body;
}
```

Example

```
for(int i=0;i<5;i++)
{
    cout<<i;
```

}

MIPS Version

```
li $t0,0

For:

    slti $t1,$t0,5

    beq $t1,$zero,EndFor

    # Loop Body

    addi $t0,$t0,1

    j For

EndFor:
```

Notice that the logic is almost identical to the `while` loop. The difference lies in how we think about the initialization, condition, and update.

---

7. The `do-while` Loop

C++ Version

```
do
{
    cout<<"Hello";
}
while(i<5);
```

Important Difference

A `do-while` loop executes the body **at least once** before checking the condition.

MIPS Version

```
li $t0,1

Do:

    # Loop Body

    addi $t0,$t0,1

    slti $t1,$t0,6
```

```
bne $t1,$zero,Do
```

The condition is checked **after** executing the loop body.

---

8. Visual Comparison

WHILE

Condition

↓

Body

↓

Update

↓

Repeat

FOR

Initialization

↓

Condition

↓

Body

↓

Update

↓

Repeat

DO-WHILE

Body

↓

Update

↓

Condition

↓

Repeat

---

9. Program – Sum of First 10 Numbers

C++ Code

```
sum = 0;


for(i=1;i<=10;i++)
    sum += i;
```

MIPS Code

```
li $t0,1
li $t1,0

Loop:
    slti $t2,$t0,11
    beq $t2,$zero,Finish
    add $t1,$t1,$t0
    addi $t0,$t0,1
    j Loop

Finish:
li $v0,10
syscall
```

Final value:

$t1 = 55

---

10. Program – Multiplication Table of 5

```
li $t0,1

li $t1,5


Loop:

    slti $t2,$t0,11

    beq $t2,$zero,Exit

    mul $t3,$t0,$t1

    # Display result (optional)

    addi $t0,$t0,1

    j Loop
```

Exit:

Students can extend this later to print a formatted table.

---

11. Nested Loops

Example in C++:

```
for(i=1;i<=3;i++)
{
   for(j=1;j<=3;j++)
   {
      ...
   }
}
```

MIPS uses two labels:

OuterLoop:

InnerLoop:

Each loop has:

- Its own counter
- Its own condition
- Its own update
- Its own branch

---

12. Loop Execution in MARS

Demonstrate:

1. Assemble the program.
2. Click **Step**.
3. Observe:
    - `$t0` (loop counter)
    - `$t1` (sum or result)
    - Program Counter (PC)
4. Watch execution jump back to the loop label until the condition fails.

This helps students understand how loops actually execute.

---

13. Real-Life Analogy

Imagine climbing stairs.

Step 1

↓

Step 2

↓

Step 3

↓

Continue...

↓

Stop when you reach the top.

A loop repeats an action until the stopping condition is met.

---

14. Common Mistakes

Students often:

❌ Forget to update the loop variable.

Example:

j Loop

without:

addi $t0,$t0,1

This creates an **infinite loop**.

❌ Place the branch instruction in the wrong location.

Always:

Condition

↓

Branch

↓

Body

↓

Update

↓

Jump Back

❌ Reuse the same register for multiple loop counters in nested loops.

Use different registers, for example:

$t0 → Outer Loop

$t1 → Inner Loop

🎤 Lecture Summary

Today, we learned how loops are implemented in MIPS Assembly Language using comparison, branch, and jump instructions.

We translated `while`, `for`, and `do-while` loops from C/C++ into MIPS, wrote programs to print numbers, calculate sums, and generate multiplication tables, and learned how to debug loop execution in the MARS simulator.

In the next lecture, we'll start working with **arrays in MIPS Assembly Language**, where we'll learn how arrays are stored in memory, how to access elements using addresses and offsets, and how to process arrays using loops.

Thank you for watching, and I'll see you in the next lecture.
